Complete Phidata Agent Development Master Guide

Build AI Agents From Beginner to Advanced

Table of Contents

36. Final Capstone Projects

---

1. Introduction to AI Agents

What is an AI Agent?

An AI Agent is an intelligent software system that:

- Understands user input
- Thinks and reasons
- Uses tools
- Remembers information
- Makes decisions
- Performs actions
- Solves problems autonomously

Traditional AI:

```
question -> AI -> answer
```

AI Agent:

```
question -> reasoning -> tools -> memory -> actions -> answer
```

---

2. What is Phidata?

Phidata is a powerful framework for building:

- AI Agents
- Multi-Agent Systems
- RAG Applications
- Autonomous AI Systems
- AI Workflows
- Tool-Using Agents

It helps developers create production-ready AI agents quickly.

---

3. Why Use Phidata?

Advantages

1. Simple Syntax

```
from phi.agent import Agent

agent = Agent()
```

2. Tool Integration

Agents can:

- Search web
- Read files
- Use databases
- Call APIs
- Execute code

3. Memory Support

Agents remember conversations.

4. Multi-Agent Systems

Multiple agents can work together.

5. Production Ready

Supports:

- Docker
- APIs
- Databases
- Monitoring

---

4. Phidata Architecture

Main Components

```
User
  ↓
Agent
  ↓
LLM
  ↓
Tools + Memory + Knowledge
  ↓
Response
```

---

5. Installation and Environment Setup

Step 1: Install Python

Recommended Version:

```
Python 3.10+
```

Check version:

```
python --version
```

Step 2: Create Virtual Environment

Windows

```
python -m venv venv
venv\Scripts\activate
```

Linux/Mac

```
python3 -m venv venv
source venv/bin/activate
```

Step 3: Install Phidata

```
pip install phidata
```

OR with uv:

```
uv pip install phidata
```

Step 4: Install OpenAI

```
pip install openai
```

Step 5: Create API Key

Create API keys from:

- OpenAI
- Groq
- Anthropic
- Gemini

Step 6: Setup Environment Variables

Create `.env`

```
OPENAI_API_KEY=your_api_key_here
```

Step 7: Install dotenv

```
pip install python-dotenv
```

Step 8: Test Installation

Create `test.py`

```
from phi.agent import Agent

print("Phidata Installed Successfully")
```

Run:

```
python test.py
```

---

6. Understanding Core Concepts

1. Agent

The brain of the system.

```
agent = Agent()
```

---

2. Model

The LLM powering the agent.

```
model=OpenAIChat(id="gpt-4o")
```

---

3. Tools

External capabilities.

Examples:

- Web search
- Calculator
- Database
- APIs

---

4. Memory

Stores previous conversations.

---

5. Knowledge Base

Custom information source.

---

7. Your First Agent

Minimal Agent

```
from phi.agent import Agent
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o")
)

agent.print_response("Tell me about AI")
```

How It Works

Step 1

User sends message.

Step 2

Agent forwards to LLM.

Step 3

LLM generates response.

Step 4

Agent displays output.

---

8. Understanding LLMs in Phidata

OpenAI Models

```
from phi.model.openai import OpenAIChat

model = OpenAIChat(id="gpt-4o")
```

Groq Models

```
from phi.model.groq import Groq

model = Groq(id="llama3-70b-8192")
```

Anthropic Models

```
from phi.model.anthropic import Claude

model = Claude(id="claude-3-5-sonnet")
```

Gemini Models

```
from phi.model.google import Gemini

model = Gemini(id="gemini-1.5-pro")
```

---

9. Tools in Phidata

What Are Tools?

Tools give superpowers to agents.

Without tools:

```
Agent can only talk.
```

With tools:

```
Agent can search, calculate, analyze, code, fetch data.
```

9.1 Calculator Tool Example

```
from phi.agent import Agent
from phi.tools.calculator import Calculator
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[Calculator()],
    show_tool_calls=True,
)

agent.print_response("What is 234 * 56?")
```

Understanding the Code

tools=[Calculator()]

Adds calculator capability.

show_tool_calls=True

Shows when tool is used.

9.2 Web Search Agent

```
from phi.agent import Agent
from phi.tools.duckduckgo import DuckDuckGo
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGo()],
    show_tool_calls=True,
)

agent.print_response("Latest AI news")
```

9.3 Python Tool Agent

```
from phi.agent import Agent
from phi.tools.python import PythonTools
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[PythonTools()],
    show_tool_calls=True,
)

agent.print_response("Plot a sine wave")
```

---

10. Memory Systems

Why Memory Matters

Without memory:

```
Every conversation starts fresh.
```

With memory:

```
Agent remembers user preferences and previous chats.
```

Simple Memory Example

```python
from phi.agent import Agent
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    add_history_to_messages=True,
)

agent.print_response("My name is Saif")
agent.print_response("What is my name?")
```

---

11. Knowledge Bases

What is a Knowledge Base?

A custom information source.

Examples:

- PDFs
- Documents
- Company data
- Books
- Research papers

11.1 PDF Knowledge Base

Install dependencies:

```
pip install pypdf lancedb sentence-transformers
```

Create Knowledge Base

```
from phi.knowledge.pdf import PDFKnowledgeBase
from phi.vectordb.lancedb import LanceDb

knowledge_base = PDFKnowledgeBase(
    path="data/manual.pdf",
    vector_db=LanceDb(
        table_name="documents",
        uri="tmp/lancedb",
    ),
)

knowledge_base.load()
```

Create RAG Agent

```
from phi.agent import Agent
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    knowledge=knowledge_base,
    search_knowledge=True,
)

agent.print_response("Summarize the PDF")
```

12. Multi-Agent Systems

What Are Multi-Agent Systems?

Multiple specialized agents working together.

Example:

```
Research Agent
Writing Agent
Coding Agent
Review Agent
```

12.1 Team of Agents Example

```python
from phi.agent import Agent
from phi.model.openai import OpenAIChat

research_agent = Agent(
    name="Research Agent",
    role="Researches topics",
    model=OpenAIChat(id="gpt-4o"),
)

writer_agent = Agent(
    name="Writer Agent",
    role="Writes content",
    model=OpenAIChat(id="gpt-4o"),
)

team = Agent(
    team=[research_agent, writer_agent],
    model=OpenAIChat(id="gpt-4o"),
)

team.print_response("Write article about AI")
```

---

13. RAG (Retrieval Augmented Generation)

What is RAG?

RAG =

```
Retrieve Information + Generate Response
```

Instead of relying only on LLM knowledge:

```
Agent searches documents first.
```

13.1 RAG Workflow

```
User Question
      ↓
Search Vector Database
      ↓
Retrieve Relevant Chunks
      ↓
Send to LLM
      ↓
Generate Accurate Response
```

13.2 Complete RAG Example

```python
from phi.agent import Agent
from phi.model.openai import OpenAIChat
from phi.knowledge.pdf import PDFKnowledgeBase
from phi.vectordb.lancedb import LanceDb

knowledge_base = PDFKnowledgeBase(
    path="documents",
    vector_db=LanceDb(
        table_name="pdf_documents",
        uri="tmp/lancedb",
    ),
```

```
)

knowledge_base.load(recreate=False)

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    knowledge=knowledge_base,
    search_knowledge=True,
    markdown=True,
)

agent.print_response("Explain chapter 1")
```

---

14. Agent Teams

Real Workflow Example

Startup Team

1. Market Research Agent
2. Financial Agent
3. Pitch Deck Agent
4. Marketing Agent
5. Coding Agent

Example

```
from phi.agent import Agent
from phi.model.openai import OpenAIChat

researcher = Agent(
    name="Researcher",
    role="Finds market trends",
    model=OpenAIChat(id="gpt-4o"),
)

developer = Agent(
    name="Developer",
    role="Writes code",
    model=OpenAIChat(id="gpt-4o"),
)
```

```
marketer = Agent(
    name="Marketer",
    role="Creates marketing strategies",
    model=OpenAIChat(id="gpt-4o"),
)

startup_team = Agent(
    team=[researcher, developer, marketer],
    model=OpenAIChat(id="gpt-4o"),
)

startup_team.print_response(
    "Create AI fitness startup plan"
)
```

---

15. Web Search Agents

Install Tools

```
pip install duckduckgo-search
```

AI Research Agent

```
from phi.agent import Agent
from phi.tools.duckduckgo import DuckDuckGo
from phi.model.openai import OpenAIChat

research_agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGo()],
    instructions=[
        "Always provide sources",
        "Search latest information",
    ],
    show_tool_calls=True,
)

research_agent.print_response(
    "Latest AI trends in 2026"
)
```

16. File Processing Agents

Reading Files

```
from phi.agent import Agent
from phi.tools.file import FileTools
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[FileTools()],
    show_tool_calls=True,
)

agent.print_response(
    "Read report.txt and summarize"
)
```

17. Database Agents

SQL Agent

```
from phi.agent import Agent
from phi.tools.sql import SQLTools
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[
        SQLTools(
            db_url="sqlite:///students.db"
        )
    ],
    show_tool_calls=True,
)

agent.print_response(
    "Show top 5 students"
)
```

18. Finance Agents

Stock Analysis Agent

```
from phi.agent import Agent
from phi.tools.yfinance import YFinanceTools
from phi.model.openai import OpenAIChat

finance_agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[
        YFinanceTools(
            stock_price=True,
            analyst_recommendations=True,
            company_info=True,
        )
    ],
)

finance_agent.print_response(
    "Analyze Tesla stock"
)
```

---

19. AI Research Agents

Advanced Research System

```
from phi.agent import Agent
from phi.tools.duckduckgo import DuckDuckGo
from phi.model.openai import OpenAIChat

research_agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGo()],
    instructions=[
        "Search deeply",
        "Provide citations",
        "Think step by step",
        "Summarize clearly",
    ],
    markdown=True,
)
```

```
research_agent.print_response(
    "Research quantum computing"
)
```

---

20. Autonomous Agents

What Are Autonomous Agents?

Agents that:

- Plan tasks
- Break down problems
- Use tools independently
- Iterate until completion

Example

```
from phi.agent import Agent
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions=[
        "Break problems into steps",
        "Think carefully",
        "Verify answers",
    ],
)

agent.print_response(
    "Create a complete startup roadmap"
)
```

---

21. Voice Agents

Voice AI Workflow

```
Speech → Text → Agent → Text → Speech
```

Required Libraries

```
pip install speechrecognition pyttsx3
```

Voice Assistant Example

```
import speech_recognition as sr
import pyttsx3

from phi.agent import Agent
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o")
)

recognizer = sr.Recognizer()
engine = pyttsx3.init()

with sr.Microphone() as source:
    print("Speak...")
    audio = recognizer.listen(source)

text = recognizer.recognize_google(audio)

response = agent.run(text)

engine.say(response.content)
engine.runAndWait()
```

---

22. Vision Agents

What Are Vision Agents?

Agents that can:

- Understand images
- Analyze screenshots
- Read charts
- Detect objects

Vision Example

```
from phi.agent import Agent
from phi.model.openai import OpenAIChat

vision_agent = Agent(
    model=OpenAIChat(id="gpt-4o")
)

vision_agent.print_response(
    "Describe this image",
    images=["image.jpg"],
)
```

---

23. API Integration

Why APIs Matter

Agents become powerful when connected to:

- Weather APIs
- Payment APIs
- Maps APIs
- AI APIs
- SaaS platforms

Custom API Tool

```
import requests

from phi.tools import Toolkit

class WeatherTools(Toolkit):
    def __init__(self):
        super().__init__(name="weather_tools")
        self.register(self.get_weather)

    def get_weather(self, city: str):
        url = f"https://api.weatherapi.com/{city}"
        response = requests.get(url)
        return response.text
```

Use Custom Tool

```
from phi.agent import Agent
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[WeatherTools()],
)
```

---

24. Production Deployment

Production Architecture

```
Frontend
   ↓
FastAPI Backend
   ↓
Phidata Agents
   ↓
Database + APIs
```

---

25. Docker Setup

Dockerfile

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

Build Docker Image

```
docker build -t ai-agent .
```

Run Container

```
docker run -p 8000:8000 ai-agent
```

---

26. FastAPI Integration

Install FastAPI

```
pip install fastapi uvicorn
```

API Server

```
from fastapi import FastAPI
from pydantic import BaseModel

from phi.agent import Agent
from phi.model.openai import OpenAIChat

app = FastAPI()

agent = Agent(
    model=OpenAIChat(id="gpt-4o")
)

class RequestData(BaseModel):
    message: str

@app.post("/chat")
def chat(data: RequestData):
    response = agent.run(data.message)

    return {
        "response": response.content
    }
```

Run Server

```
uvicorn main:app --reload
```

---

27. Streamlit Integration

Install Streamlit

```
pip install streamlit
```

Streamlit App

```
import streamlit as st

from phi.agent import Agent
from phi.model.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o")
)

st.title("AI Agent")

user_input = st.text_input("Ask something")

if st.button("Send"):
    response = agent.run(user_input)
    st.write(response.content)
```

Run Streamlit

```
streamlit run app.py
```

---

28. Security Best Practices

Never Expose API Keys

Wrong:

```
OPENAI_API_KEY="abc123"
```

Correct:

```
import os

key = os.getenv("OPENAI_API_KEY")
```

Use Environment Variables

```
OPENAI_API_KEY=your_key
```

---

29. Cost Optimization

Use Cheaper Models for Simple Tasks

Examples:

- Small tasks → cheap model
- Complex reasoning → expensive model

Reduce Token Usage

Strategies:

- Short prompts
- Summarized memory
- Smaller context

---

30. Debugging and Monitoring

Show Tool Calls

```
show_tool_calls=True
```

Enable Markdown

```
markdown=True
```

Logging Example

```
import logging

logging.basicConfig(level=logging.INFO)
```

---

31. Real-World Projects

Project 1: AI Coding Assistant

Features

- Write code
- Debug code
- Explain code
- Generate APIs

Example

```
from phi.agent import Agent
from phi.tools.python import PythonTools
from phi.model.openai import OpenAIChat

coding_agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[PythonTools()],
    instructions=[
        "Write clean code",
        "Add comments",
        "Optimize performance",
    ],
)
```

Project 2: AI Startup Co-Founder

Architecture

```
CEO Agent
Research Agent
Marketing Agent
Finance Agent
Developer Agent
```

Complete Example

```python
from phi.agent import Agent
from phi.model.openai import OpenAIChat
from phi.tools.duckduckgo import DuckDuckGo

research_agent = Agent(
    name="Research Agent",
    role="Analyzes market",
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGo()],
)

finance_agent = Agent(
    name="Finance Agent",
    role="Calculates revenue",
    model=OpenAIChat(id="gpt-4o"),
)

developer_agent = Agent(
    name="Developer Agent",
    role="Creates MVP roadmap",
    model=OpenAIChat(id="gpt-4o"),
)

startup_agent = Agent(
    team=[
        research_agent,
        finance_agent,
        developer_agent,
    ],
    model=OpenAIChat(id="gpt-4o"),
)
```

```
startup_agent.print_response(
    "Build AI healthcare startup"
)
```

Project 3: AI Customer Support Agent

Features

- Answer customer questions
- Search FAQs
- Handle tickets
- Escalate issues

Example

```
from phi.agent import Agent
from phi.model.openai import OpenAIChat

support_agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions=[
        "Be polite",
        "Solve customer issues",
        "Escalate critical problems",
    ],
)
```

---

32. Advanced Architectures

32.1 Planner + Executor Architecture

Flow

```
User Query
    ↓
Planner Agent
```

```
    ↓
Task Breakdown
    ↓
Executor Agents
    ↓
Final Result
```

Example

```
planner = Agent(
    name="Planner",
    role="Breaks tasks into subtasks",
)

executor = Agent(
    name="Executor",
    role="Executes subtasks",
)
```

32.2 Reflection Architecture

Flow

```
Generate Answer
     ↓
Critique Answer
     ↓
Improve Answer
```

---

33. Best Practices

1. Give Clear Instructions

Bad:

```
Help user.
```

Good:

```
Provide concise technical answers with examples.
```

---

2. Limit Tool Access

Only provide necessary tools.

---

3. Use Specialized Agents

Instead of one giant agent:

```
Use multiple specialized agents.
```

---

4. Add Memory Carefully

Too much memory increases costs.

---

5. Use RAG for Accuracy

Avoid hallucinations.

---

34. Common Errors and Solutions

Error 1: API Key Missing

Error

```
OPENAI_API_KEY not found
```

Solution

Check `.env`

```
OPENAI_API_KEY=your_key
```

Error 2: Tool Not Working

Solution

Install dependency.

Example:

```
pip install duckduckgo-search
```

Error 3: Import Errors

Solution

Update packages.

```
pip install --upgrade phidata
```

Error 4: Memory Issues

Solution

Reduce context size.

---

35. Complete Learning Roadmap

Beginner Level

Learn:

- Python basics
- APIs
- Environment variables
- Basic agents

Projects:

- Chatbot
- Calculator agent
- Web search agent

Intermediate Level

Learn:

- RAG
- Memory
- Tool creation
- APIs

Projects:

- PDF assistant
- Research assistant
- Coding assistant

Advanced Level

Learn:

- Multi-agent systems
- Autonomous workflows
- Deployment
- Scaling

Projects:

- AI startup cofounder
- AI operating system
- Autonomous business agents

---

36. Final Capstone Projects

Project 1: Jarvis AI Assistant

Features:

- Voice interaction
- Web search
- Memory
- Coding
- File management
- Automation

Project 2: Autonomous Research Lab

Features:

- Multiple research agents
- Citation generation
- PDF analysis
- Report generation

Project 3: AI SaaS Platform

Features:

- User authentication
- AI agents
- Billing
- Dashboard
- APIs
- Analytics

Complete Production Folder Structure

```
project/
│
├── agents/
│   ├── research_agent.py
│   ├── coding_agent.py
│   └── finance_agent.py
│
├── tools/
│   ├── weather_tools.py
│   └── custom_tools.py
│
├── knowledge/
│   └── documents/
│
├── api/
│   └── main.py
│
├── frontend/
│   └── app.py
│
├── .env
├── requirements.txt
├── Dockerfile
└── README.md
```

Complete Requirements File

```
phidata
openai
python-dotenv
duckduckgo-search
fastapi
uvicorn
streamlit
pypdf
lancedb
sentence-transformers
sqlalchemy
yfinance
```

```
speechrecognition
pyttsx3
```

How to Think Like an AI Agent Engineer

1. Think in Workflows

Instead of:

```
Single response
```

Think:

```
Planning → Reasoning → Tool Use → Validation
```

---

2. Divide Problems

Large problems should be split into:

- Research
- Analysis
- Execution
- Validation

---

3. Design Specialized Agents

Each agent should:

- Have a single responsibility
- Be optimized for one task
- Use relevant tools

Complete Example: Ultimate AI Assistant

```
from phi.agent import Agent
from phi.model.openai import OpenAIChat
from phi.tools.duckduckgo import DuckDuckGo
from phi.tools.python import PythonTools
from phi.tools.calculator import Calculator

assistant = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[
        DuckDuckGo(),
        PythonTools(),
        Calculator(),
    ],
    instructions=[
        "Be helpful",
        "Think step by step",
        "Use tools when needed",
        "Provide accurate answers",
    ],
    show_tool_calls=True,
    markdown=True,
)

assistant.print_response(
    "Research AI trends and calculate market growth"
)
```

Final Advice

To Become an Expert:

Build Daily

Practice by creating:

- Mini agents
- Tools
- APIs
- Workflows

Study Agent Architectures

Learn:

- ReAct
- Chain of Thought
- Reflection
- Planning
- Multi-agent collaboration

Build Real Products

Examples:

- AI SaaS
- AI automation systems
- AI startup tools
- AI research systems

What You Can Build After This

After mastering this guide you can build:

- AI SaaS products
- Autonomous AI systems
- AI employees
- Research assistants
- Startup copilots
- Coding agents
- AI operating systems
- Business automation agents
- Voice assistants
- AI workflows
- AI startup founders

Next Technologies to Learn

After Phidata:

1. LangGraph
2. CrewAI
3. AutoGen

4. OpenAI Agents SDK
5. MCP Protocol
6. Vector Databases
7. Kubernetes
8. Redis
9. Docker
10. FastAPI

Final Mission

Build these 5 projects in order:

1. Chatbot Agent
2. Research Agent
3. RAG PDF Assistant
4. Multi-Agent Startup Team
5. Autonomous AI SaaS Platform

If you complete these projects yourself, you will become capable of building production-grade AI agent systems.

End of Documentation